

บทที่ 2

ทฤษฎีและหลักการ

2.1 ไมโครคอนโทรลเลอร์ MCS-51

ไมโครคอนโทรลเลอร์ (Microcontroller) เป็นชื่อของอุปกรณ์อิเล็กทรอนิกส์แบบหนึ่งที่บรรจุความสามารถมากมายไม่ว่าจะเป็นหน่วยประมวลผล หน่วยคำนวณทางคณิตศาสตร์และลอจิก วงจรรับสัญญาณอินพุต วงจรขับสัญญาณทางเอาต์พุต หน่วยความจำ วงจรกำเนิดสัญญาณนาฬิกา ทำให้ไมโครคอนโทรลเลอร์สามารถนำไปประยุกต์ใช้งานแทนวงจรอิเล็กทรอนิกส์ที่ซับซ้อนได้เป็นอย่างดี โดยช่วยลดจำนวนของอุปกรณ์และขนาดของระบบลง ในขณะที่มีขีดความสามารถสูงขึ้นภายใต้งบประมาณที่เหมาะสม

ไมโครคอนโทรลเลอร์มาจากคำสองคำคือ “ไมโคร” (Micro) ซึ่งหมายถึงไมโครโปรเซสเซอร์ (Microprocessor) ซึ่งเป็นอุปกรณ์ประมวลผลข้อมูลขนาดเล็ก ซึ่งภายในประกอบด้วยหน่วยประมวลผลกลางหรือซีพียู (CPU : Central Processing Unit) หน่วยคำนวณทางคณิตศาสตร์และลอจิก (ALU : Arithmetic Logic Unit) วงจรเชื่อมต่อหน่วยความจำ และวงจรเชื่อมต่อสัญญาณนาฬิกา อีกคำหนึ่งคือ คำว่า “คอนโทรลเลอร์”(Controller) หมายถึงอุปกรณ์ควบคุม ดังนั้นไมโครคอนโทรลเลอร์จึงเป็นอุปกรณ์ที่ใช้ในการควบคุม โดยที่สามารถเขียนโปรแกรมเพื่อกำหนดรูปแบบการควบคุมได้อย่างอิสระ

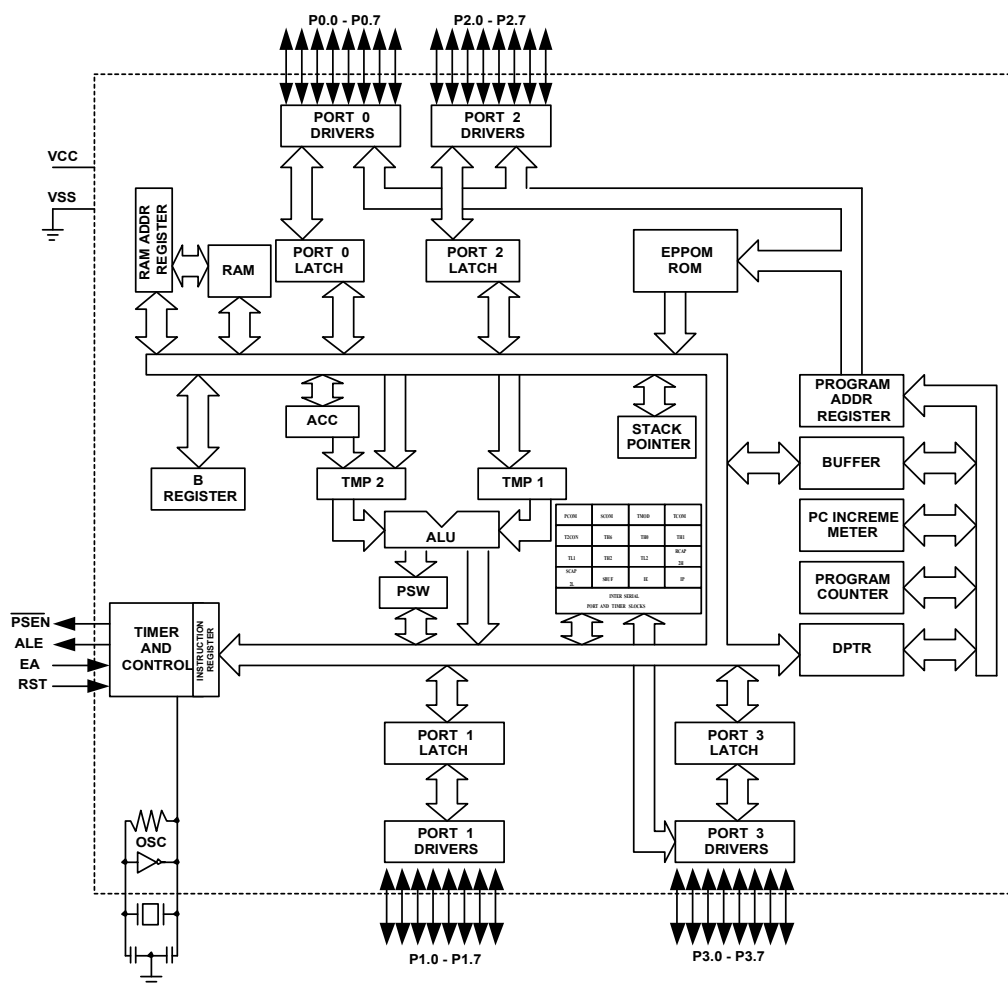
2.1.1 คุณสมบัติทางเทคนิคของไมโครคอนโทรลเลอร์ ตระกูล MCS-51

- เป็นไมโครคอนโทรลเลอร์ที่ใช้ซีพียูขนาด 8 บิต
- ภายในมีหน่วยความจำโปรแกรมเป็นแบบแฟลชสามารถลบและเขียนใหม่ได้
- หน่วยความจำข้อมูลพื้นฐานเป็นหน่วยความจำแบบแรม ในบางเบอร์จะมีหน่วยความจำแบบอีอีพรอมเพิ่มเติม

- ขาพอร์ตเป็นแบบ 2 ทิศทาง สามารถใช้งานได้เป็นทั้งอินพุตและเอาต์พุต
- มีวงจรสื่อสารอนุกรมแบบฟูลดูเพล็กซ์
- ไทมเมอร์/เคาน์เตอร์ขนาด 16 บิต อย่างน้อย 2 ตัว
- สามารถรองรับแหล่งกำเนิดอินเตอร์รัปต์ได้ 6 ประเภท
- สามารถขยายหน่วยความจำภายนอกเพิ่มเติมได้สูงสุด 64 กิโลไบต์
- มีวงจรถ่ายสัญญาณนาฬิกาภายในชิป
- มีวงจรสื่อสารอนุกรมแบบ SPI
- มีวอตช์ด็อกไทมเมอร์ในตัว

2.1.2 โครงสร้างของ MCS-51

ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีด้วยกันหลายเบอร์ขึ้นกับโครงสร้างภายใน บางเบอร์จะมีหน่วยความจำภายในเป็นแบบ ROM บางเบอร์เป็นแบบ EPROM บางเบอร์มี RAM ภายใน 128 ไบต์ บางเบอร์มี 256 ไบต์ เป็นต้น รายละเอียดโครงสร้างหลักของไมโครคอนโทรลเลอร์ MCS-51 แสดงดังรูปที่ 2.1



รูปที่ 2.1แสดงโครงสร้างภายในของ MCS-51

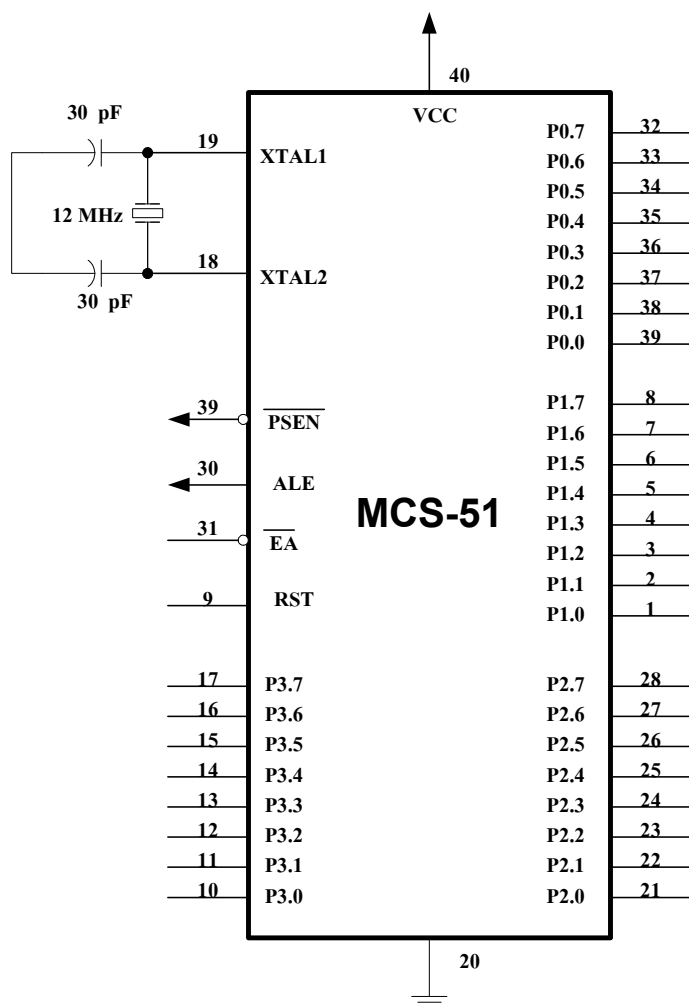
2.1.3 การจัดขาต่าง ๆ ของ MCS-51

ไอซี ไมโครคอนโทรลเลอร์ MCS-51 โครงสร้าง IC เป็นแบบ DIP มีขาทั้งหมด 40 ขาโดยขาต่าง ๆ จะใช้เป็นขาพอร์ตอินพุต, เอาต์พุต, ขาสัญญาณควบคุม, ขาดำหนดหน่วยความจำ และขาข้อมูลดังรูปที่ 2.2 ความหมายของขาต่าง ๆ มีดังนี้

2.1.3.1 พอร์ต 0 (Port 0) พอร์ต 0 ได้แก่ขาที่ 32-39 ของ MCS-51 สามารถใช้เป็นอินพุต เอาต์พุตได้ นอกจากนี้ในการติดต่อกับหน่วยความจำภายนอกยังใช้เป็นขา Address Bus และ Data Bus อีกด้วย

2.1.3.2 พอร์ต 1 (Port 1) พอร์ต 1 ได้แก่ขาที่ 1-8 เป็นพอร์ต 8 บิต สามารถอ้างทีละบิตได้ คือ P1.0, P1.1,..., P1.7

2.1.3.3 พอร์ต 2 (Port 2) พอร์ต 2 ได้แก่ขาที่ 21-28 จะใช้งาน 2 หน้าที คือใช้เป็นพอร์ต 8 บิตกับใช้เป็นขาแอดเดรส 8 บิตในการอ้างหน่วยความจำภายนอก



รูปที่ 2.2 แสดงขาต่าง ๆ ของไมโครคอนโทรลเลอร์ MCS-51

2.1.3.4 พอร์ต 3 (Port 3) พอร์ต 3 ได้แก่ขาที่ 10-17 จะใช้งานสองหน้าที่คือ เป็นพอร์ตอินพุตและเอาต์พุต และใช้เป็นขาควบคุมต่าง ๆ ดังตารางที่ 2.1

บิต	ชื่อ	หน้าที่พิเศษ
P3.0	RXD	ใช้รับข้อมูลทางพอร์ตอนุกรม
P3.1	TXD	ใช้ส่งข้อมูลทางพอร์ตอนุกรม
P3.2	INT0	อินเทอร์รัปต์ภายนอกหมายเลข 0
P3.3	INT1	อินเทอร์รัปต์ภายนอกหมายเลข 1
P3.4	T0	ตัวจับเวลา/ตัวนับ ตัวที่ 0
P3.5	T1	ตัวจับเวลา/ตัวนับ ตัวที่ 1
P3.6	$\overline{WR}$	สัญญาณเขียนข้อมูลหน่วยความจำภายนอก
P3.7	$\overline{RD}$	สัญญาณอ่านข้อมูลหน่วยความจำภายนอก

ตารางที่ 2.1 แสดงหน้าที่ของขาต่าง ๆ ในพอร์ต 3

2.1.3.5 PSEN (Program Store Enable) ขา PSEN เป็นขาที่ส่งสัญญาณออก ได้แก่ ขา 29 ขานี้จะแอคทีฟเมื่อ MCS-51 ต้องการอ่าน Code โปรแกรมภายนอก โดยปกติถ้าหน่วยความจำภายนอกเป็น EPROM ขา PSEN จะต่อกับขา Output Enable (OE) ของ EPROM

2.1.3.6 ALE (Address Latch Enable) เนื่องจากพอร์ต 0 สามารถใช้เป็นขาอ้างตำแหน่ง และขาข้อมูล MCS-51 จึงมีขา ALE ได้แก่ขา 30 Multiplex สัญญาณ Address bus ของ Port 0 ในการใช้งานระบบ MCS-51 นั้นจำเป็นต้องมีอุปกรณ์มาต่อกับ Port 0 ที่ทำหน้าที่ Latch สัญญาณ Address Bus เมื่อ MCS-51 ต้องการติดต่อกับหน่วยความจำภายนอก MCS-51 จะส่งสัญญาณ Address Bus ออกมาก่อนทาง Port 0 จากนั้นจะส่งสัญญาณ ALE มา Latch อุปกรณ์ภายนอก ให้เก็บค่า Address Bus ของ Port 0 ไว้เพื่อใช้ Port เป็น Data Bus ต่อไป

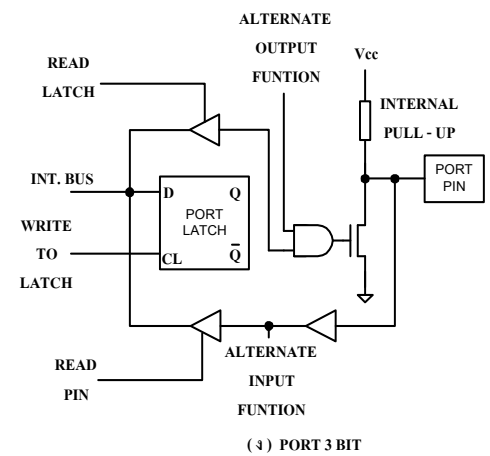
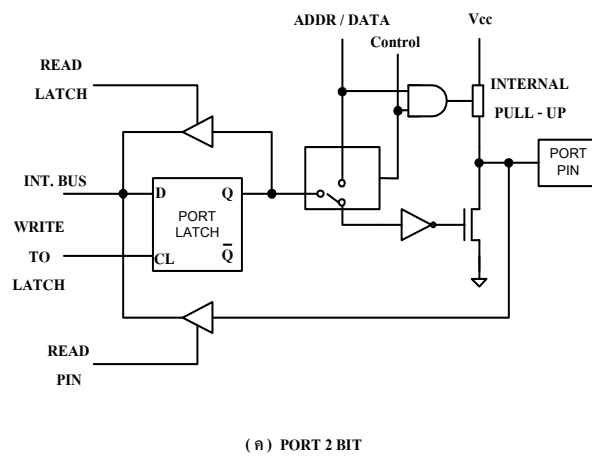
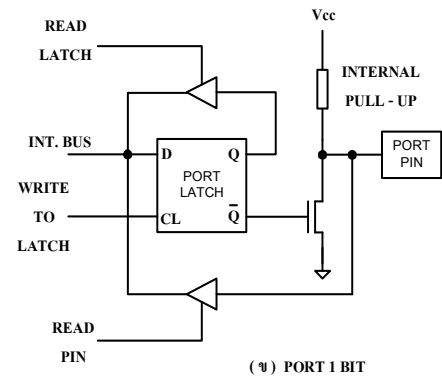
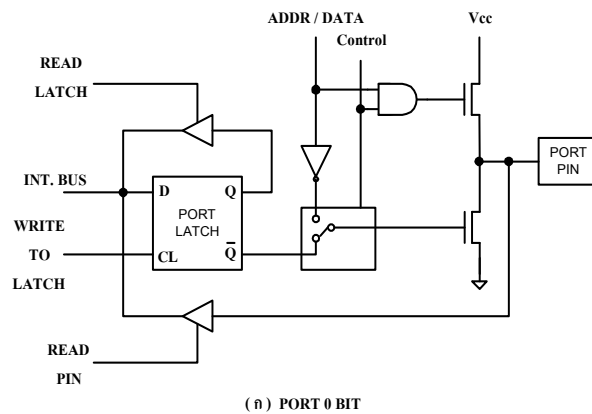
2.1.3.7 EA (External Access) EA ได้แก่ขาที่ 31 ถ้าขานี้เป็นลอจิก “1” จะใช้เบอร์ 8051/8052 เพื่อบอกว่าให้อ่านโปรแกรมจากหน่วยความจำโปรแกรมภายใน แต่ถ้าเป็นลอจิก “0” จะบอกให้ MCS-51 ทำโปรแกรมโดย อ่านจากหน่วยความจำโปรแกรมภายนอก (ถ้าขา EA เป็น “0” ขา PSEN จะแอคทีฟ) ถ้าหากเป็นเบอร์ 8031 หรือ 8032 ขา EA จะเป็น “0” เสมอ เพราะไม่มีโปรแกรมหน่วยความจำภายใน แต่ถ้าใช้เบอร์ 8051/8052 ซึ่งมีหน่วยความจำโปรแกรมภายในและให้ขา EA เป็น “0” ซึ่งจะ Disabled ROM ภายในและจะอ่านโปรแกรมจาก EPROM ภายนอกแทน

2.1.3.8 RST (Reset) ขา RST ได้แก่ขา 9 จะใช้ในการรีเซ็ต MCS-51 โดยจะให้ขานี้เป็นลอจิก “1” อย่างน้อย 2 Machine Cycles จึงจะรีเซ็ตระบบได้

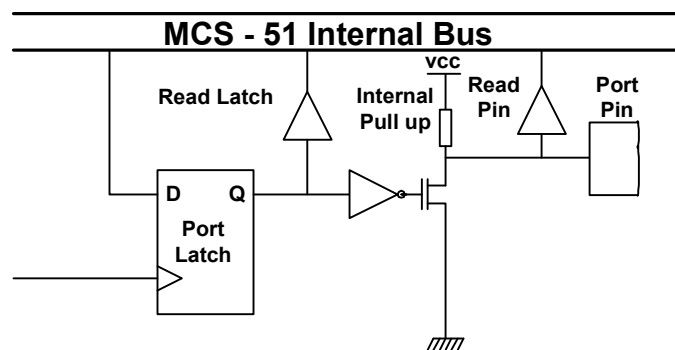
2.1.4 โครงสร้างของพอร์ตอินพุตเอาต์พุต (I/O Port Structure)

ขาของพอร์ตจะแสดงโครงสร้างภายในได้ ดังรูปที่ 2.3 โดยจะมีโครงสร้างเป็น Field-effect Transistor ต่ออยู่กับขาภายนอก และมีความต้านทานต่อ Pull-up อยู่สำหรับพอร์ต 1, 2, 3 แต่ถ้าเป็นพอร์ต 0 จะไม่มีตัวต้านทาน Pull-up ภายใน เพราะต้องใช้เป็นขา Address Bus และ Data Bus

พอร์ตนี้สามารถใช้เป็นอินพุตเอาต์พุตกับอุปกรณ์ภายนอกได้ ในการอ่านข้อมูลจากพอร์ตจะอ่านได้สองแบบ คือ Read Latch และ Read Pin โดย Read Latch หมายถึงการอ่านข้อมูลที่ถูกลatch เอาไว้เข้าสู่บัตภายในของ MCS-51 เช่น การทำคำสั่ง CPL P1.5 แต่ถ้าเป็นการ Read Pin จะเป็นการใช้พอร์ตเป็นอินพุต โดยจะอ่านค่าจากขาของไอซีเข้าสู่บัตภายใน การอ่านแบบ Read Latch และ Read Pin จะมีสัญญาณควบคุมที่บัพเฟอร์ดังรูปที่ 2.4



รูปที่ 2.3 แสดงโครงสร้างพอร์ตทั้ง 4 ของ MCS-51



รูปที่ 2.4 แสดงการต่อพอร์ตเข้ากับระบบบัสภายในของ MCS-51

2.1.5 พอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51

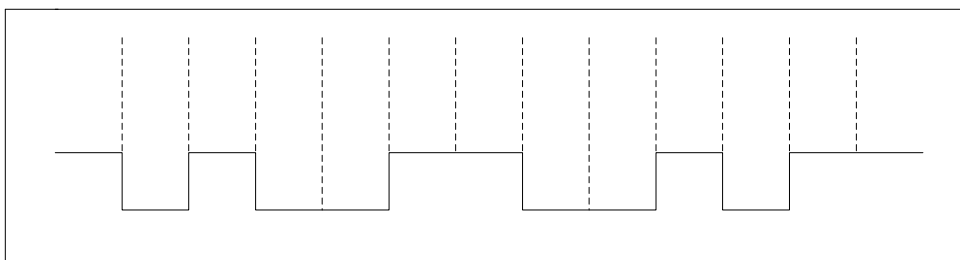
ไมโครคอนโทรลเลอร์ตระกูล MCS-51 มีวงจรสื่อสารอนุกรมแบบฟูลดูเพล็กซ์ 1 ชุด (วงจรสื่อสารแบบฟูลดูเพล็กซ์ หมายถึง วงจรสื่อสารที่สามารถทำการรับและส่งข้อมูลในลักษณะ 2 ทิศทาง ได้ในเวลาเดียวกัน) โดยใช้ขาสัญญาณของพอร์ต 3 คือ ขา P3.0 เป็นขารับข้อมูลเข้าหรือ RxD และขา P3.1 เป็นขาส่งข้อมูลออกหรือ TxD โดยวงจรสื่อสารข้อมูลแบบอนุกรมของไมโครคอนโทรลเลอร์ตระกูล MCS-51 แบบแฟลชเป็นแบบอะซิงโครนัส ปกติแล้วพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 จะใช้ในการติดต่อสื่อสารกับพอร์ตอนุกรมของคอมพิวเตอร์ โดยใช้มาตรฐาน RS-232 แต่ในปัจจุบันสามารถติดต่อกันในมาตรฐาน RS-422 หรือ RS-485 ได้แล้ว โดยใช้ไอซีพิเศษที่ทำหน้าที่ในการแปลงสัญญาณการสื่อสารดังกล่าว

2.1.5.1 การสื่อสารข้อมูลแบบอะซิงโครนัส

การสื่อสารข้อมูลแบบอะซิงโครนัสคือการรับและส่งข้อมูลโดยไม่จำเป็นต้องมีสัญญาณนาฬิกาพร้อมด้วย แต่จะทำการกำหนดอัตราเร็วในการรับและส่งข้อมูลให้มีค่าเท่ากัน เรียกอัตราเร็วนี้ว่า อัตราบอด หรือ บอดเรต (baud rate) มีหน่วยเป็น บิตต่อวินาที (bit per second : bps)

รูปแบบของข้อมูลที่ใช้ในการรับส่งแบบอะซิงโครนัสประกอบด้วย 4 ส่วนด้วยกันคือ

1. บิตเริ่มต้น (start bit) มีขนาด 1 บิต
2. บิตข้อมูลแบบอนุกรม มีขนาด 8 บิต
3. บิตตรวจสอบพาริตี (parity bit) มีขนาด 1 บิตหรือไม่มี
4. บิตปิดท้ายหรือบิตหยุด (stop bit) มีขนาด 1 บิต



รูปที่ 2.5 แสดงรูปแบบข้อมูลอนุกรมแบบอะซิงโครนัส

รูปที่ 2.5 แสดงรูปแบบของข้อมูล อนุกรมแบบอะซิงโครนัส เมื่อไม่มีการส่งข้อมูล ขา DATA จะมีสถานะลอจิก “1” เรียกสถานะนี้ว่า สถานะหยุดรอ (waiting stage) การเริ่มต้นส่งข้อมูลจะเริ่มจากการให้ขา DATA มีลอจิก “0” ด้วยช่วงระยะเวลา 1 บิต เรียกบิตนี้ว่า บิตเริ่มต้น (start bit) จากนั้นบิตข้อมูลจะถูกส่งออกไป โดยเริ่มจากบิตที่มีนัยสำคัญต่ำสุดหรือบิต LSB ก่อน ข้อมูลที่ต้องการส่งมีจำนวน 8 บิต ตามด้วยบิตพาริตี (parity bit) ซึ่งใช้ในการตรวจสอบความผิดพลาดที่เกิดขึ้นจากการส่งข้อมูล บิตสุดท้ายที่จะส่งคือ บิตปิดท้ายหรือบิตหยุด (stop bit) โดยจะเป็นการทำให้ขา DATA มีสถานะลอจิก “1” อีกครั้งด้วยระยะเวลาอย่างน้อย 1 บิต, 1.5 บิต หรือ 2 บิต เพื่อเป็นการแสดงว่าสิ้นสุดข้อมูลแล้ว

อัตราความเร็วในการรับและส่งข้อมูลของการรับส่งข้อมูลแบบอะซิงโครนัสหรืออัตราบอดหรือบอดเรตที่ใช้สำหรับพอร์ตอนุกรม RS-232 มีด้วยกันหลายค่า ได้แก่ 110, 150, 300, 600, 1200, 2400, 4800, 9600 และ 19200 บิตต่อวินาที โดยมีค่าเพิ่มมากขึ้นตามเทคโนโลยีของคอมพิวเตอร์ เนื่องจากอัตราบอดคือค่าของจำนวนบิตที่สามารถส่งได้ใน 1 วินาที สมมติว่า ข้อมูลอนุกรมมีขนาด 8 บิต ไม่มีการตรวจสอบพาริตี มีบิตเริ่มต้น 1 บิต และบิตปิดท้าย 1 บิต ความยาวของข้อมูล 1 ไบต์จะเท่ากับความยาว 10 บิต ถ้าใช้บอดเรตในการส่งข้อมูลเท่ากับ 9,600 บิตต่อวินาที ก็สามารถรับส่งข้อมูลได้ด้วยความเร็ว 960 ไบต์ต่อวินาที

การตรวจสอบพาริตีสามารถกำหนดให้เป็นแบบคี่(odd),แบบคู่(even) หรือไม่มีการตรวจสอบพาริตีก็ได้ การตรวจสอบแบบพาริตีคี่(หรือคู่)คือ การนับจำนวนลอจิก “1” ของข้อมูลขนาด 1 ไบต์และของบิตพาริตีว่ามีจำนวนรวมเป็นเลขคี่(หรือคู่)หรือไม่ ถ้าใช่แสดงว่าข้อมูลที่ส่งมาถูกต้อง ยกตัวอย่าง ข้อมูลที่จะทำการส่งมีขนาด 8 บิต มีค่าเท่ากับ 99H หรือ 10011001B จะเห็นได้ว่าข้อมูลในไบต์นี้มีจำนวนลอจิก“1”จำนวน 4 ตัวซึ่งเป็นเลขคู่ ดังนั้นถ้ากำหนดการตรวจสอบค่าพาริตีเป็นแบบคู่ ค่าของบิตพาริตีจะต้องมีลอจิกเป็น“0” แต่ถ้ากำหนดการตรวจสอบพาริตีเป็นแบบคี่ ค่าของบิตพาริตีจะต้องเป็น“1” เพื่อให้จำนวนลอจิก “1” ข้อมูล 1 ไบต์รวมทั้งในบิตพาริตีมีจำนวนรวมเป็นคี่

บิตพาริตีจะถูกสร้างขึ้นจากภาคส่งข้อมูลของ UART(Universal Asynchronous Receiver Transmitter: เป็นอุปกรณ์ที่ใช้ในการรับและส่งข้อมูลอนุกรม) ซึ่งทางภาครับจะต้องกำหนดคุณสมบัติการตรวจสอบพาริตีที่ตรงกันเอาไว้ว่าตรวจสอบพาริตีคี่หรือพาริตีคู่ จากนั้นภาครับของ UART จะทำการตรวจสอบค่าพาริตีที่เกิดขึ้นว่าเป็นคู่หรือเป็นคี่ โดยการนับจำนวนลอจิก 1 ทั้งหมดรวมทั้งบิตพาริตีด้วย ถ้ากำหนดพาริตีไว้เป็นคู่แต่อ่านค่าตัวเลขในการนับออกมาได้ตัวเลขเป็นคี่ ทางภาครับจะแสดงข้อผิดพลาดออกมาให้ผู้ใช้งานทราบ กระบวนการดังกล่าวเป็นวิธีการตรวจสอบความผิดพลาดที่เกิดขึ้นในการรับส่งข้อมูลที่ง่ายที่สุด แต่มันสามารถตรวจสอบได้เมื่อมีบิตข้อมูลที่ทำการรับส่งผิดพลาดเพียงบิตเดียวเท่านั้น ถ้าข้อมูลที่ทำการส่งมีบิตที่มีการผิดพลาดมากกว่า 1 บิต การตรวจสอบด้วยวิธีนี้จะไม่ได้ผล สำหรับการตั้งพาริตีบิตเป็น NONE นั้นทั้งภาครับและภาคส่ง จะไม่มีการตรวจสอบพาริตี

2.1.5.2 รีจิสเตอร์ที่เกี่ยวข้องกับการทำงานของพอร์ตอนุกรมใน MCS-51

ในการทำงานของวงจรพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 มีรีจิสเตอร์ที่ต้องเกี่ยวข้องอยู่ 2 ตัว มีรายละเอียดดังต่อไปนี้

1. รีจิสเตอร์บัฟเฟอร์ของพอร์ตอนุกรมหรือ SBUF (Serial Data Buffer Register)

มีแอดเดรสอยู่ที่ 99H ในพื้นที่ของรีจิสเตอร์ฟังก์ชันพิเศษหรือ SFR มีขนาด 8 บิต แบ่งเป็น 2 ส่วนคือ รีจิสเตอร์บัฟเฟอร์สำหรับส่งข้อมูล (transmit buffer register) และรีจิสเตอร์บัฟเฟอร์สำหรับรับข้อมูล (receive buffer register) เมื่อมีการเขียนข้อมูลมายังรีจิสเตอร์ SBUF ข้อมูลนั้นจะถูกส่งต่อไปยังบัฟเฟอร์สำหรับส่งข้อมูล เพื่อส่งออกจากไมโครคอนโทรลเลอร์ผ่านทางขาTxD หรือ P3.1 ในกรณีที่มีการอ่านข้อมูลจากรีจิสเตอร์ SBUF ข้อมูลจะถูกส่งผ่านไปยังรีจิสเตอร์บัฟเฟอร์สำหรับรับข้อมูลเพื่อส่งต่อไปยังไมโครคอนโทรลเลอร์ต่อไป สำหรับการรับข้อมูลอนุกรมจากภายนอกนั้นจะผ่านมาจากขา RxD หรือ P3.0 ของไมโครคอนโทรลเลอร์ MCS-51 แบบแฟลช

2. รีจิสเตอร์ควบคุมการทำงานของพอร์ตอนุกรมหรือ SCON

(Serial Port Control Register)

เป็นรีจิสเตอร์ขนาด 8 บิต มีแอดเดรสอยู่ที่ 98H ในพื้นที่ของรีจิสเตอร์ SFR สามารถเข้าถึงได้ในระดับบิต มีรายละเอียดการทำงานดังนี้

บิต 7	บิต 6	บิต 5	บิต 4	บิต 3	บิต 2	บิต 1	บิต 0
SM0	SM1	SM2	SM3	SM4	SM5	SM6	SM7

รูปที่ 2.6 แสดงรีจิสเตอร์ควบคุมการทำงานของพอร์ตอนุกรม

SM0-SM1 (Serial port mode bit 0-1) : ใช้ในการเลือกโหมดการทำงานของพอร์ตอนุกรมภายในไมโครคอนโทรลเลอร์

SM2 : ใช้ในการอินทิเกรตการสื่อสารในแบบมัลติโพรเซสเซอร์ (multiprocessor) ในการทำงานของโหมด 2 และ 3 ของพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 ในโหมด 2 และ 3 ถ้าบิตนี้เป็น "1" บิต RI จะไม่แอกติฟถ้าบิตที่ 9 ที่รับเข้ามาเป็น "0" (ข้อมูลบิตที่ 9 เก็บไว้ที่บิต RB8) ในการทำงานโหมด 1 ถ้าบิตนี้ถูกเซตบิต RI จะไม่แอกติฟถ้ายังไม่รับบิตหยุด ส่วนในโหมด 0 บิตนี้ไม่มีการใช้งาน

REN(Enable serial reception): ใช้ในการอินทิเกรตการรับข้อมูลพอร์ตอนุกรม ทำการเซตและเคลียร์ด้วยกระบวนการทางซอฟต์แวร์ ถ้าต้องการรับข้อมูลต้องเซตบิตนี้ให้เป็น "1"

TB8: ใช้สำหรับเก็บข้อมูลที่บิต 9 ที่ต้องการส่งออกไปขณะทำงานในโหมด 2 และ 3 ของพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 เซตและเคลียร์ด้วยกระบวนการทางซอฟต์แวร์

RB8: ใช้สำหรับข้อมูลบิตที่ 9 ที่เข้ามาขณะทำงานในโหมด 2 และ 3 ของพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 แต่ถ้าหากพอร์ตอนุกรมทำงานอยู่ในโหมด 1 และบิต SM2 เป็น "0" ข้อมูลที่บิต RB8 คือข้อมูลของบิตหยุด (stop bit) สำหรับการทำงานในโหมด 0 บิตนี้จะไม่ใช้งาน บิต RB8 นี้สามารถเซตและเคลียร์ด้วยกระบวนการทางซอฟต์แวร์

TI(Transmit Interrupt flag) : ใช้ในการแสดงการเกิดอินเตอร์รัปต์เมื่อมีการส่งข้อมูลออกจากพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 สามารถเซตได้ด้วยกระบวนการทางฮาร์ดแวร์ เมื่อทำการส่งข้อมูลบิตที่ 8 เรียบร้อยแล้วในการทำงานโหมด 0 ส่วนการทำงานโหมดอื่น บิตนี้จะเซตเมื่อมีการเริ่มต้นส่งบิตหยุดออกไป การเคลียร์บิตนี้ต้องใช้กระบวนการทางซอฟต์แวร์เท่านั้น

RI(Receive Interrupt flag) : ใช้ในการแสดงการเกิดอินเตอร์รัปต์เมื่อมีการรับข้อมูลเข้าจากพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 สามารถเซตได้ด้วยกระบวนการทางฮาร์ดแวร์ เมื่อทำการรับข้อมูลบิตที่ 8 เรียบร้อยแล้วในการทำงานโหมด 0 ส่วนการทำงานโหมดอื่น บิตนี้จะเซตเมื่อมีการรับบิตหยุดของข้อมูลอนุกรมไปได้ครึ่งทางแล้ว ยกเว้นกรณีที่บิต SM2 มีการเซต บิตนี้จะเซตได้ก็ต่อเมื่อการรับบิตหยุดหรือบิตที่ 9 เกิดขึ้นอย่างสมบูรณ์แล้ว การเคลียร์บิตนี้ต้องใช้กระบวนการทางซอฟต์แวร์เท่านั้น

2.1.5.3 โหมดการทำงานของพอร์ตอนุกรมใน MCS-51

พอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 แบบแฟลชสามารถเลือกโหมดการทำงานได้ถึง 4 โหมดคือ

1. โหมด 0 เป็นการกำหนดให้พอร์ตอนุกรมทำงานในลักษณะซีพรีจิสเตอร์
2. โหมด 1 เป็นการกำหนดให้เป็น UART ขนาด 8 บิต สามารถเลือกอัตราบอดได้
3. โหมด 2 เป็นการกำหนดให้เป็น UART ขนาด 9 บิต โดยมีอัตราบอดคงที่
4. โหมด 3 เป็นการกำหนดให้เป็น UART ขนาด 9 บิต สามารถเลือกอัตราบอดได้

การเลือกโหมดการทำงานของวงจรพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51 กระทำได้โดยการกำหนดข้อมูลให้แก่บิต SMO และ SM1 ในรีจิสเตอร์ SCON

2.1.5.4 อัตราบอดของพอร์ตอนุกรมในไมโครคอนโทรลเลอร์ MCS-51

1. โหมด 0

อัตราบอดของโหมดนี้มีค่าคงที่ โดยสามารถคำนวณได้จากสูตร

อัตราบอดในโหมด 0 = ความถี่ของสัญญาณนาฬิกา/12 มีหน่วยเป็นบิตต่อวินาที

2. โหมด 1 และ 3

ทั้งสองโหมดนี้สามารถเลือกแหล่งกำเนิดอัตราบอดได้ 2 แหล่งคือ จากอัตราโอเวอร์โฟลวของไทเมอร์ 1 และ 2 สำหรับอัตราบอดเมื่อใช้การโอเวอร์โฟลวของไทเมอร์ 1 จะต้องใช้ค่าของบิต SMOD ในรีจิสเตอร์ PCON มาพิจารณาประกอบด้วย โดยสามารถหาค่าอัตราบอดได้จาก

$$\text{อัตราบอด} = (2^{\text{ค่าของบิต SMOD}} / 32) \times \text{อัตราโอเวอร์โฟลวของไทเมอร์ 1}$$

อัตราบอด (บิตต่อวินาที : bps)	ความถี่ สัญญาณนาฬิกา	SMOD	ไทเมอร์ 1		
			C/T	โหมด	ค่ารีโหลด
โหมด 0 : สูงสุด 1 MHz	12 MHz	x	x	x	x
โหมด 2 : สูงสุด 375k	12 MHz	1	x	x	x
โหมด 1,3 : 62.5 K	12 MHz	1	0	2	FFH
19.2K (19,200)	11.0592 MHz	1	0	2	FDH
9.6K (9,600)	11.0592 MHz	0	0	2	FDH
4.8K (4,800)	11.0592 MHz	0	0	2	FAH
2.4K (2,400)	11.0592 MHz	0	0	2	F4H
1.2K (1,200)	11.0592 MHz	0	0	2	E8H
137.5	11.0592 MHz	0	0	2	1DH
110	6 MHz	0	0	2	72H
110	12 MHz	0	0	1	FEEDH

ตารางที่ 2.2 แสดงการเลือกอัตราบอดของวงจรพอร์ตอนุกรมภายในไมโครคอนโทรลเลอร์ MCS-51

ถ้าหากในไทมเมอร์ 1 ไม่ได้เปิดการอินเตอร์รัปต์ไว้ สามารถคำนวณหาอัตราบอดได้จาก

$$\text{อัตราบอด} = (2^{\text{ค่าในรีจิสเตอร์ SMOD}}/32) \times (\text{ความถี่สัญญาณนาฬิกา}/\{12 \times [256 - (\text{TH1})]\})$$

ตารางที่ 2-2 แสดงการกำหนดอัตราบอดโดยใช้ไทมเมอร์ 1

กรณีที่ 1 ใช้ไทมเมอร์ 2 ในการกำหนดอัตราบอด โดยกำหนดให้ไทมเมอร์ 2 ทำงานในโหมดกำหนดอัตราบอด (baud rate generator) สามารถคำนวณหาอัตราบอดได้จาก

$$\text{อัตราบอด} = \text{อัตราโอเวอร์โฟลวของไทมเมอร์ 2}/16 \text{ มีหน่วยเป็นบิตต่อวินาที}$$

ถ้าหากกำหนดให้ไทมเมอร์ 2 ทำงานในโหมดไทมเมอร์หรือเคาน์เตอร์ตามปกติ สามารถคำนวณหาอัตราบอดได้จาก

$$\text{อัตราบอด} = \text{ความถี่สัญญาณนาฬิกา}/(32 \times (65536 - (\text{RCAP2H}, \text{RCAP2L})))$$

โดยที่ (RCAP2H, RCAP2L) เป็นค่าของรีจิสเตอร์ RCAP2H และ RCAP2L มีขนาด 16 บิตไม่คิดเครื่องหมาย

3. โหมด 2

ในโหมดนี้การกำหนดอัตราบอดจะขึ้นอยู่กับค่าของบิต SMOD ในรีจิสเตอร์ PCON ถ้า SMOD เป็น "0" อัตราบอดจะเท่ากับ 1/64 ของความถี่สัญญาณนาฬิกา ในกรณีที่ SMOD เป็น "1" อัตราบอดจะเท่ากับ 1/32 ของความถี่สัญญาณนาฬิกา สามารถแสดงเป็นสูตรคำนวณทางคณิตศาสตร์ได้ดังนี้

$$\text{อัตราบอด} = (2^{\text{ค่าของบิต SMOD}}/64) \times \text{ความถี่สัญญาณนาฬิกา}$$

2.1.5.5 การกำหนดค่าของไทมเมอร์เพื่อเลือกอัตราบอด

ในการใช้งานพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 สิ่งที่ต้องให้ความสนใจมากที่สุดประการหนึ่งคือ อัตราการถ่ายทอข้อมูล หรือ อัตราบอด ซึ่งการกำหนดอัตราบอดนั้นจะขึ้นอยู่กับค่าความถี่ของสัญญาณนาฬิกาเป็นหลัก สำหรับโหมดการทำงานของพอร์ตอนุกรมที่สามารถเลือกอัตราบอดได้อย่างอิสระคือโหมด 1 และ 3 โดยกำหนดได้จากอัตราการเกิดโอเวอร์โฟลวของไทมเมอร์ 1 หากไทมเมอร์ 1 มีการเกิดโอเวอร์โฟลวในอัตราที่สูงมากอัตราบอดก็จะมีค่าสูงมากขึ้นตาม นั่นหมายความว่า อัตราในการถ่ายทอข้อมูลจะสูงมาก สามารถถ่ายทอข้อมูลได้อย่างรวดเร็ว ดังนั้นการกำหนดค่าและโหมดการทำงานของไทมเมอร์ 1 จึงเป็นสิ่งที่มีความสำคัญต่อการเปลี่ยนแปลงของอัตราบอดด้วย

ในการใช้ไทมเมอร์ 1 เพื่อกำหนดอัตราบอดในโหมด 1 และ 3 ของพอร์ตอนุกรมจะต้องกำหนดให้ไทมเมอร์ ทำงานในโหมด 2 หรือโหมด 8 บิตแบบตั้งค่าการนับอัตโนมัติ ในกรณีการกำหนดค่ารีโหลดให้แกรีจิสเตอร์ TH1 จะเป็นตัวแปรหลักที่ใช้ในการกำหนดอัตราบอดให้แก่พอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51

เริ่มต้นด้วยการเคลียร์บิต SMOD ซึ่งเป็นบิต 7 ของรีจิสเตอร์ PCON ให้เป็น "0" ค่าของกรรีโหลดให้แก่ TH1 สามารถคำนวณได้จาก

$$\text{TH1} = 256 - ((\text{ค่าความถี่ของคริสตัล}/384)/\text{อัตราบอด})$$

แต่ถ้าบิต SMOD เกิดจากการเซต จะเป็นการเปิดการทวิคูณของอัตราบอด ดังนั้นการกำหนดค่าให้แก่ TH1 จึงต้องคำนวณจาก

$$\text{TH1} = 256 - ((\text{ค่าความถี่ของคริสตัล}/192)/\text{อัตราบอด})$$

ยกตัวอย่าง ถ้าหากในไมโครคอนโทรลเลอร์ AT89C51 ใช้คริสตอล 11.0592 MHz ต้องการกำหนดอัตราบอดของพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ไว้ที่ 19,200 บิตต่อวินาที ในกรณีที่ไม้อินาเบิลการทวิคูณของอัตราบอด ค่ารีโหนดของไมโครคอนโทรลเลอร์จะเท่ากับ

$$\begin{aligned} TH1 &= 256 - ((\text{ค่าความถี่ของคริสตอล}/384)/\text{อัตราบอด}) \\ &= 256 - ((11059200/384)/19200) \\ &= 256 - (28800/19200) \\ &= 256 - 1.5 = 254.5 \end{aligned}$$

เนื่องจากผลลัพธ์ที่ได้เป็นค่าที่ไม่ใช่จำนวนเต็ม ถ้าหากกำหนดค่าของ TH1 เป็น 254 เมื่อทำการแทนค่าเพื่อคำนวณหาอัตราบอด จะได้อัตราบอดเท่ากับ 14,400 บิตต่อวินาที และถ้าหากกำหนดค่าของ TH1 เป็น 255 อัตราบอดจะมีค่าเท่ากับ 28,800 บิตต่อวินาที ดังนั้นจะเห็นได้ว่าค่าของ TH1 ที่ไม่เป็นจำนวนเต็มจะไม่สามารถทำให้เกิดอัตราบอดตามต้องการได้

ทางแก้ไขคือ ให้ทำการอินาเบิลการทวิคูณอัตราบอด โดยการเซตบิต SMOD ในรีจิสเตอร์ PCON ให้เป็น “1” จากนั้นทำการแทนค่าลงในสมการหาค่า TH1 เมื่อมีการเซตบิต SMOD ได้ผลดังนี้

$$\begin{aligned} TH1 &= 256 - ((\text{ค่าความถี่ของคริสตอล}/192)/\text{อัตราบอด}) \\ &= 256 - ((11059200/192)/19200) \\ &= 256 - (57600/19200) \\ &= 256 - 3 = 253 \end{aligned}$$

นำค่าของ TH1 ที่ได้ทำการแทนค่าคำนวณหาอัตราบอดจะได้เท่ากับ 19,200 บิตต่อวินาที สามารถสรุปขั้นตอนในการเลือกอัตราบอดโดยการกำหนดค่าของไทมเมอร์ 1 ได้ดังนี้

1. กำหนดให้พอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 ทำงานในโหมด 1 หรือ 3
2. กำหนดให้ไทมเมอร์ 1 ทำงานในโหมด 2 หรือโหมด 8 บิตตั้งค่าอัตราบอด
3. กำหนดข้อมูลให้แก่ TH1 เท่ากับ 253 เพื่อให้สามารถกำเนิดอัตราบอดได้ 19,200 บิตต่อวินาที

ตามที่ต้องการ

4. ทำการเซตบิต SMOD ซึ่งเป็นบิต 7 ของรีจิสเตอร์ PCON เพื่ออินาเบิลการทวิคูณอัตราบอด

2.1.5.6 การเขียนหรือส่งข้อมูลออกจากพอร์ตอนุกรม

ข้อมูลที่ต้องการส่งออกทุกค่าต้องนำไปเก็บไว้ในรีจิสเตอร์บัฟเฟอร์ของพอร์ตอนุกรม ซึ่งก็คือ รีจิสเตอร์ SBUF ดังตัวอย่าง

```
MOV SBUF, #'A'
```

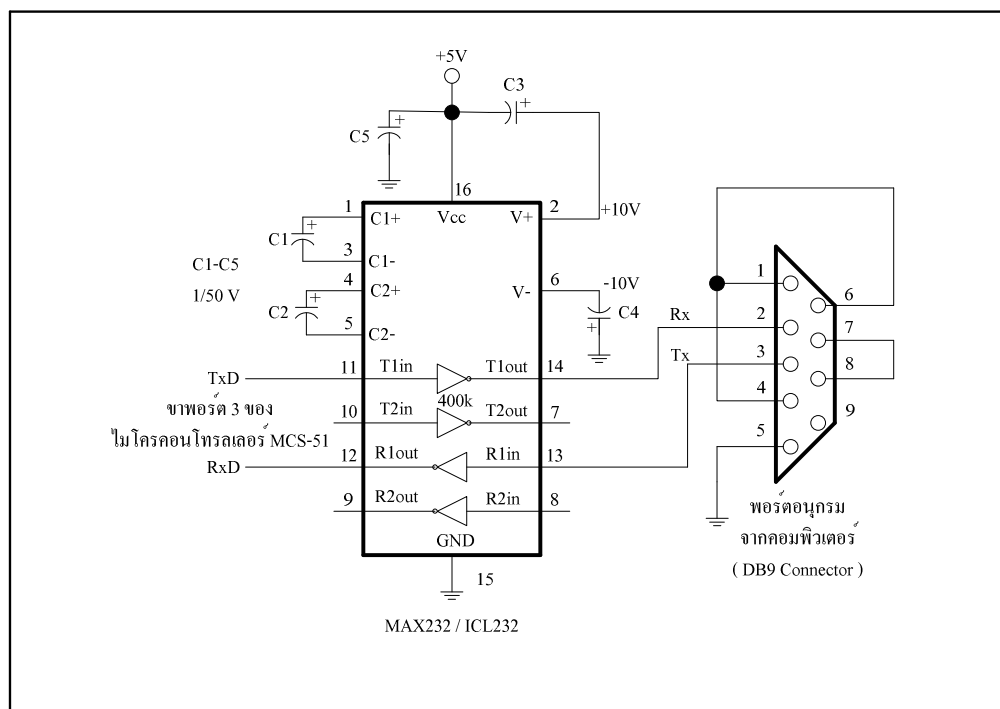
จากคำสั่งข้างต้นเป็นการส่งข้อมูลของตัวอักษร A ออกไปยังพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ อย่างไรก็ตามก่อนที่จะส่งข้อมูลทุกครั้ง ต้องแน่ใจว่าบิต TI เคลียร์ หรือ มีค่าเป็น “0” และเมื่อทำการส่งข้อมูลเรียบร้อยแล้ว ก็จะเกิดการเซตบิต TI เพื่อแจ้งให้ทราบ ดังตัวอย่างโปรแกรมต่อไปนี้

CLR TI ; เคลียร์บิต TI เพื่อเตรียมการส่งข้อมูลออก
 MOV SBUF, #'A' ; ส่งข้อมูลของตัวอักษร A ไปยังพอร์ตอนุกรม
 JNB TI, \$; รอการเซตของบิต TI เพื่อแจ้งการส่งข้อมูลที่เสร็จสมบูรณ์

2.1.5.7 การอ่านหรือรับข้อมูลจากพอร์ตอนุกรม

การรับข้อมูลจากพอร์ตอนุกรมสามารถทำได้ง่ายมาก เพียงแค่ทำการตรวจสอบว่าบิต RI เกิดการเซตขึ้นหรือไม่ ถ้าพบว่ามีบิต RI เซตเกิดขึ้นแล้ว ให้ทำการอ่านค่าจากรีจิสเตอร์ SBUF โดยต้องทำการโอนย้ายข้อมูลผ่านทางแอดคิวมูลเตอร์หรือรีจิสเตอร์ A ดังตัวอย่าง

CLR RI ; เคลียร์บิต RI เพื่อเตรียมการรับข้อมูล
 JNB RI, \$; รอคอยการเซตของบิต RI อันเป็นการแจ้งให้ทราบว่า การรับข้อมูล
 ; เสร็จสมบูรณ์และมีข้อมูลที่เกิดขึ้นที่รีจิสเตอร์ SBUF
 MOV A, SBUF ; อ่านค่าจากรีจิสเตอร์ โดยการโอนย้ายข้อมูลผ่านทางรีจิสเตอร์ A
 CLR RI ; หลังจากทำการอ่านข้อมูลเรียบร้อยแล้ว ต้องทำการเคลียร์บิต RI เสมอ



รูปที่ 2.7 แสดงวงจรเชื่อมต่อ MAX232 หรือ ICL232 เข้ากับพอร์ตอนุกรมของคอมพิวเตอร์และไมโครคอนโทรลเลอร์

2.1.5.8 การเชื่อมต่อกับพอร์ตอนุกรมของคอมพิวเตอร์

การใช้งานวงจรพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 มักนิยมใช้ในการติดต่อเพื่อแลกเปลี่ยนข้อมูลกับคอมพิวเตอร์ผ่านทางพอร์ตอนุกรมในมาตรฐาน RS-232 เป็นส่วนใหญ่ แต่เนื่องจากระดับสัญญาณของพอร์ตอนุกรม RS-232 มีระดับตั้งแต่ 3 ถึง 12 V ในขณะที่ระดับสัญญาณของไมโครคอนโทรลเลอร์ MCS-51 อยู่ในระดับทีทีแอล ดังนั้นจึงไม่สามารถเชื่อมต่อพอร์ตอนุกรมของไมโครคอนโทรลเลอร์ MCS-51 เข้ากับพอร์ตอนุกรมของคอมพิวเตอร์ได้โดยตรง จึงต้องอาศัยการเชื่อมต่อผ่านไอซีพิเศษที่ทำหน้าที่แปลงระดับสัญญาณ

ไอซีที่ทำหน้าที่ในการแปลงระดับสัญญาณนี้จะต้องทำการแปลงข้อมูลส่งของไมโครคอนโทรลเลอร์ MCS-51 จากระดับทีทีแอลไปเป็นระดับของ RS-232 และทำการแปลงข้อมูลที่ได้รับจากคอมพิวเตอร์จากระดับของ RS-232 เป็นระดับทีทีแอลเพื่อให้สามารถถ่ายทอดไปยังไมโครคอนโทรลเลอร์ MCS-51 ได้อย่างสมบูรณ์ ไอซีดังกล่าวมีด้วยกันหลายเบอร์จากผู้ผลิต อาทิ MAX232 จาก MAXIM หรือ ICL232 จาก HARRIS เป็นต้น ในรูปที่ 2.7 แสดงส่วนวงจรของการต่อกับพอร์ตอนุกรมของคอมพิวเตอร์และไมโครคอนโทรลเลอร์

2.2 สเต็ปปีงมอเตอร์ (Stepping Motor)

สเต็ปปีงมอเตอร์ เป็นมอเตอร์ที่ขับเคลื่อนด้วยพัลส์ ลักษณะการขับเคลื่อน จะหมุนรอบแกนได้ 360 องศา มีลักษณะไม่ต่อเนื่อง แต่มีลักษณะเป็นสเต็ป โดยแต่ละสเต็ปจะขับเคลื่อนได้ 1,1.5,1.8 หรือ 2 องศา แล้วแต่ละโครงสร้างของมอเตอร์และลักษณะที่ นำมอเตอร์ไปใช้ ตัวอย่างงานที่ต้องการตำแหน่งแม่นยำ เช่น ระบบขับเคลื่อนหัวแม่พิมพ์ในเครื่องพิมพ์ (PRINTER)ระบบขับเคลื่อนหัวอ่านในเครื่องอ่านบันทึกแถบ ระบบขับเคลื่อนตำแหน่งของปากกาใน X-Y PLOTTER เป็นต้น

2.2.1 ลักษณะและโครงสร้างของสเต็ปปีงมอเตอร์

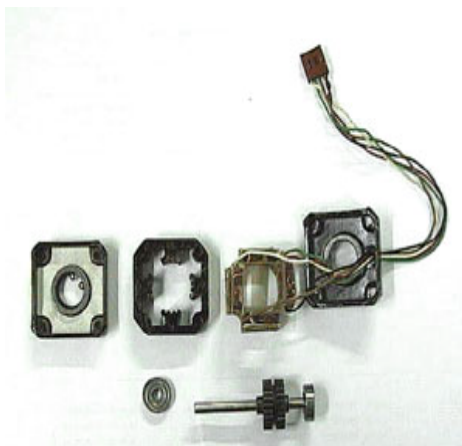
สเต็ปปีงมอเตอร์มีลักษณะดังรูป



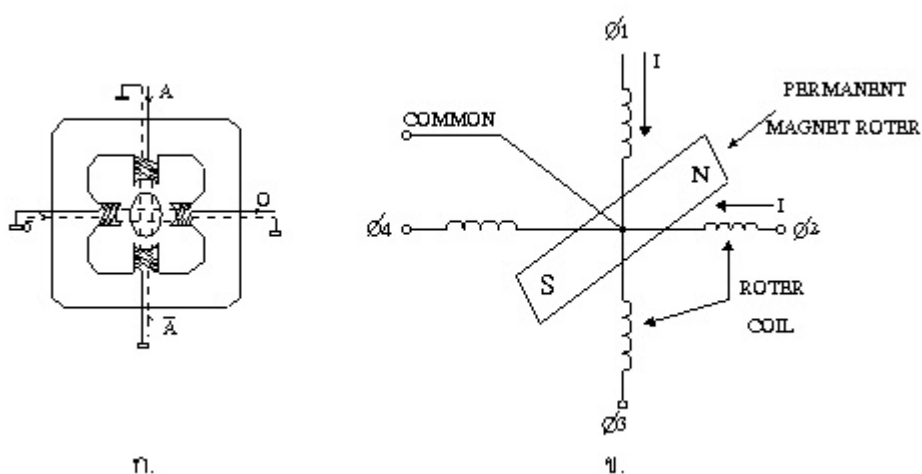
รูปที่ 2.8 แสดงสเต็ปปีงมอเตอร์แบบมีสาย 5 เส้น



รูปที่ 2.9 แสดงมอเตอร์แบบมีสาย 6 เส้น



รูปที่ 2.10 แสดงโครงสร้างของสเต็ปปีงมอเตอร์



รูป 2.11 แสดง (ก) โครงสร้าง (ข) วงจรเทียบเท่า (equivalent circuit) ของมอเตอร์ ชนิด 4 ขด

2.2.2 ชนิดของสเต็ปปีงมอเตอร์

2.2.2.1 แบบแม่เหล็กถาวร (PERMANENT MAGNET-PM) สเต็ปปีงมอเตอร์แบบ PM จะมีสเตเตอร์ (STATOR) ที่พันขดลวดไว้หลายๆ โพล โดยมีโรเตอร์ (ROTOR) เป็นรูปทรงกระบอกฟันเลื่อย และโรเตอร์ทำด้วยแม่เหล็กถาวร เพื่อป้อนไฟกระแสตรงให้กับขดสเตเตอร์ ทำให้เกิดแรงแม่เหล็กไฟฟ้าผลักต่อโรเตอร์และ ทำให้มอเตอร์หมุน มอเตอร์แบบ PM จะเกิดแรงดูดยึดให้โรเตอร์หยุดอยู่กับที่ แม้จะไม่ได้ป้อนไฟเข้าขดลวด

2.2.2.2 แบบแปรค่ารีลักแตนซ์ (VARIABLE RELUCTANCE- VR) สเต็ปปีงมอเตอร์แบบ VR จะมีการหมุนโรเตอร์ได้อย่างอิสระ แม้จะไม่ได้จ่ายไฟให้โรเตอร์ ทำจากสารเฟอร์โรแมกเนติกกำลังอ่อน มีลักษณะเป็นฟันเลื่อย รูปทรงกระบอก โดยจะมีความสัมพันธ์โดยตรงกับจำนวนโพลในสเตเตอร์ แรงบิดที่เกิดขึ้นจะหมุนโรเตอร์ไปในเส้นทางของอำนาจแม่เหล็กที่มีค่ารีลักแตนซ์ต่ำที่สุด ตำแหน่งที่จะเกิดแน่นอนและมีเสถียรภาพแต่จะเกิดขึ้นได้หลายๆ จุด ดังนั้นเมื่อป้อนไฟเข้าขดลวดต่างๆ ในมอเตอร์แตกต่างกันไป ก็ทำให้มอเตอร์หมุนไปตำแหน่งต่างๆ กัน โรเตอร์ของ VR จะมีความเฉื่อยของโรเตอร์น้อยจึงมีความเร็วรอบสูงกว่ามอเตอร์แบบ PM

2.2.2.3 แบบผสม (HYBRID-H) สเต็ปปั๊มมอเตอร์แบบ H จะเป็นลูกผสมของ VR กับ PM โดยจะมีสเตเตอร์คล้ายกับที่ใช้ใน VR โรเตอร์มีหมวกหุ้ม ปลายซึ่งมีลักษณะของสารแม่เหล็กที่มีกำลังสูง โดยการควบคุมขนาดรูปร่างของหมวกแม่เหล็กอย่างดีจะทำให้ได้มุม การหมุนและจำนวนครั้งในการหมุนน้อยและแม่นยำ ข้อดีก็คือ ให้แรงบิดสูงและมีขนาดกระทัดรัด และให้แรงจลนศาสตร์โรเตอร์นิ่งกับที่ตอนไม่จ่ายไฟ

2.2.3 การกระตุ้นและควบคุมการหมุนของสเต็ปปั๊มมอเตอร์

การกระตุ้นและควบคุมการหมุนของมอเตอร์ให้เคลื่อนที่ไปแต่ละสเต็ปทำได้โดยจ่ายกำลังไฟฟ้าไปยังขดลวดแต่ละขดบนสเตเตอร์ ซึ่งต้องป้อนเป็นแบบซีควนเชียล(Sequential)ในรูปแบบที่ถูกต้องด้วย สามารถแบ่งได้เป็น 3 รูปแบบคือ

2.2.3.1 แบบหนึ่งเฟสหรือแบบฟูลสเต็ป(Full step) เป็นการกระตุ้นโดยทำการกระตุ้นขดลวดทีละขดในเวลาหนึ่งไล่เรียงถัดกันไป ทำให้ในการกระตุ้นรูปแบบนี้จึงมีขดลวดเพียงขดลวดเดียวในเวลาหนึ่งที่ถูกระตุ้นเท่านั้น วงจรแบบเวฟจึงมีราคาถูกและง่าย ขั้นตอนการทำงานต่างๆ แสดงดังตารางที่ 2.3

สเต็ปที่	เฟสที่ 1	เฟสที่ 2	เฟสที่ 3	เฟสที่ 4
1	ทำงาน	-	-	-
2	-	ทำงาน	-	-
3	-	-	ทำงาน	-
4	-	-	-	ทำงาน

ตารางที่ 2.3 แสดงการควบคุมสเต็ปปั๊มมอเตอร์แบบหนึ่งเฟสหรือแบบฟูลสเต็ป(Full step)

2.2.3.2 แบบ 2 เฟส เป็นการกระตุ้นคล้ายแบบหนึ่งเฟส แต่จะทำการกระตุ้นโดยจ่ายกำลังไฟฟ้าไปที่ขดลวด 2 ขด ที่อยู่ใกล้กันในเวลาเดียวกัน และเรียงถัดไปเช่นเดียวกับแบบเวฟ การกระตุ้นสเต็ปปั๊มมอเตอร์แบบนี้สามารถเพิ่มแรงบิดได้มากกว่าแบบเวฟ โรเตอร์จะเคลื่อนที่ด้วยแรงดึงอย่างเต็มแรงจาก 2 ขดลวดที่ถูกกระตุ้นพร้อมกัน และต่อไปด้วยแรงดึงจากอีก 2 ขดลวดถัดไป สำหรับข้อเสียคือต้องใช้กำลังไฟฟ้ามากขึ้น ขั้นตอนการทำงานต่างๆ แสดงดังตารางที่ 2.4

สเต็ปที่	เฟสที่ 1	เฟสที่ 2	เฟสที่ 3	เฟสที่ 4
1	ทำงาน	ทำงาน	-	-
2	-	ทำงาน	ทำงาน	-
3	-	-	ทำงาน	ทำงาน
4	ทำงาน	-	-	ทำงาน

ตารางที่ 2.4 แสดงการควบคุมสเต็ปปั๊มมอเตอร์แบบ 2 เฟส

2.2.3.3 แบบครึ่งสเต็ป เป็นรูปแบบการผสมผสานระหว่างการกระตุ้นแบบเวฟและแบบ 2 เฟส เพื่อเพิ่มจำนวนของสเต็ปต่อรอบอีกเท่าตัวหนึ่ง ในระบบนี้จะทำการกระตุ้นเรียงกันไปเป็นลำดับดังนี้ เริ่มจากขดลวดที่ 1, 1 และ 2, 2, 2 และ 3, 3, 3 และ 4, 4, 4 และ 1 แล้ววนกลับมายังขดลวดที่ 1 แรงบิดที่ได้จากการกระตุ้นแบบนี้จะเพิ่มมากขึ้นอีก เพราะช่วงสเต็ปมีระยะสั้นลง แต่ละสเต็ปเกิดแรงดึงจาก ขดลวด 2 ขดที่ถูกกระตุ้นพร้อมกัน ความถูกต้องของตำแหน่งมีเพิ่มมากขึ้น แต่ต้องพึงระวังเรื่อง เมื่อกระตุ้น ให้ทำงานในรูปแบบนี้จะต้องหมุนถึง 2 สเต็ป จึงจะได้ระยะเท่ากับ 1 สเต็ปเต็มของการควบคุมใน 2 แบบ แรก สำหรับแหล่งจ่ายไฟฟ้าต้องใช้ขนาดเท่ากับแบบ 2 เฟสเป็นอย่างน้อยจึงจะพอ ขั้นตอนการทำงานต่างๆ แสดงดังตารางที่ 2.5

สเต็ปที่	เฟสที่ 1	เฟสที่ 2	เฟสที่ 3	เฟสที่ 4
1	ทำงาน	-	-	-
2	ทำงาน	ทำงาน	-	-
3	-	ทำงาน	-	-
4	-	ทำงาน	ทำงาน	-
5	-	-	ทำงาน	-
6	-	-	ทำงาน	ทำงาน
7	-	-	-	ทำงาน
8	ทำงาน	-	-	ทำงาน

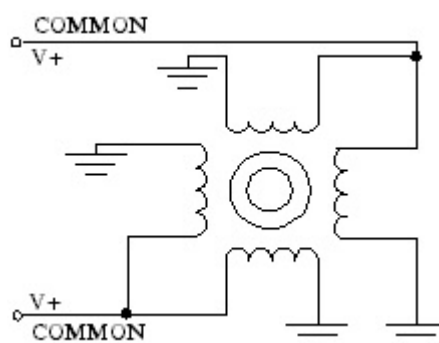
ตารางที่ 2.5 แสดงการควบคุมสเต็ปปั๊มมอเตอร์แบบครึ่งสเต็ป

2.2.4 การตรวจสอบหาสาย COMMON และสาย GROUND ของ STEPPING แบบ PM

(แบบแกนโรเตอร์เป็นแม่เหล็กถาวร)

โดยทั่วไป SP MOTOR แบบ PM จะมีอยู่ 2 ชนิด

- 1) ชนิดที่เป็น COMMON ภายนอก SP MOTOR แบบนี้มีสายอยู่ 6 เส้น ดังรูปที่ 2.12

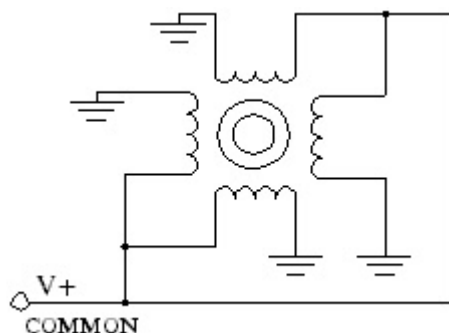


รูปที่ 2.12 แสดงสเต็ปปั๊มมอเตอร์ ชนิดมีสาย 6 เส้น

จากรูปที่ 2.12 สายที่เป็น COMMON มี 2 สาย และสายที่เป็น GROUND มี 4 สาย

สาย COMMON 1 เส้น จะ DRIVE GROUND 2 เส้น ในการเช็คให้ใช้มิเตอร์วัดหาสายที่เป็น COMMON ก่อนโดยการตั้ง RANGE ของมิเตอร์ที่ R*1 จับที่สายทีละคู่ ถ้าหากวัดสาย COMMON เทียบกับสาย GROUND ได้ถูกต้อง ค่าความต้านทานที่อ่านได้จะน้อย แต่ถ้าวัดผิดสาย คือวัดสาย GROUND เทียบกับ GROUND ค่าความต้านทานที่อ่านได้จะสูงกว่า แต่ถ้าวัดสาย COMMON เทียบกับสาย GROUND ที่ไม่ใช่คู่กันแล้ว เข็มมิเตอร์ก็จะไม่กระดิก ให้ทดลองวัดเปรียบเทียบกันทีละคู่ ก็จะทราบว่าสายใดเป็นสาย COMMON สายใดเป็นสาย GROUND

2) ชนิดที่เป็น COMMON ภายใน SP MOTOR แบบนี้มีสายอยู่ 5 เส้น ดังรูปที่ 2.13



รูปที่ 2.13 แสดงสเต็ปมอเตอร์ ชนิดมีสาย 5 เส้น

จากรูปที่ 2.13 สายที่เป็น COMMON มี 1 สาย และสายที่เป็น GROUND มี 4 สาย

ในการวัดให้ทำแบบเดียวกับการวัด SP MOTOR ชนิด COMMON ภายนอก แตกต่างกันเพียงแบบ COMMON ภายใน สาย COMMON 1 เส้น DRIVE สาย GROUND 4 เส้น ดังนั้นหากสายเส้นใดเมื่อวัดเทียบกับสายเส้นอื่น แล้วมีค่าความต้านทานน้อยที่สุดสายเส้นนั้นเป็นสาย COMMON และที่เหลืออีก 4 เส้นจะเป็นสาย GROUND

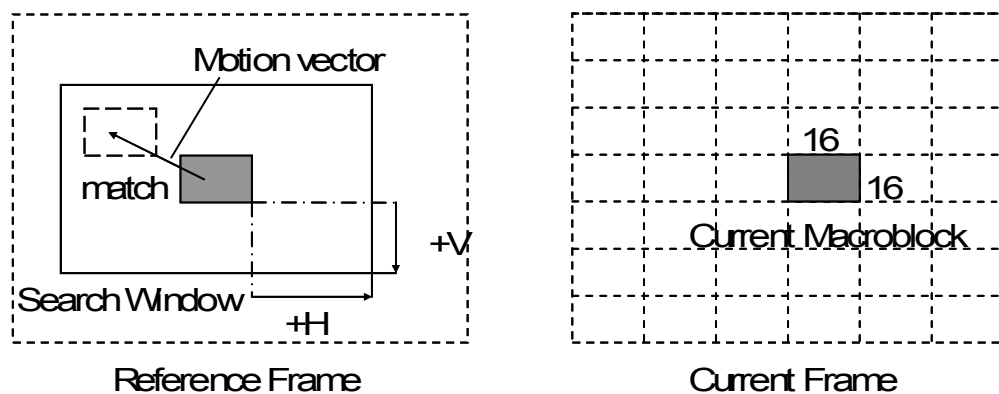
2.2.5 การเรียงเฟสของ STEPPING MOTOR แบบ PM

เมื่อเราทราบว่าสายเส้นใดเป็นสาย COMMON แล้ว แต่เรายังไม่ทราบว่าสาย GROUND เส้นใดเป็นเฟสที่ 1 เฟสที่ 2 เฟสที่ 3 และเฟสที่ 4 ในการเรียงเฟสนั้นให้ใช้มิเตอร์วัดโดยนำ V+ เข้าที่สาย COMMON วัดเทียบกับสาย GROUND เส้นใดก็ได้ 1 เส้น จะทำให้แกนโรเตอร์เคลื่อนไปข้างหน้า 1 STEP เมื่อเปลี่ยนสาย GROUND เส้นแรกเป็นเส้นที่ 2 หาก MOTOR ไม่เคลื่อนที่ไปข้างหน้าแสดงว่าการเรียงเฟสไม่ถูกต้องก็ให้วัดเทียบกับสาย GROUND เส้นใหม่ต่อไป หาก MOTOR เคลื่อนที่ไปข้างหน้าตามกัน ก็ให้วัดที่สาย GROUND เส้นต่อไปเรื่อย ๆ จะทำให้ทราบว่าสายเส้นใดเป็นเฟสแรก สายเส้นใดเป็นเฟสที่ 2 เฟสที่ 3 และเฟสที่ 4 การเรียงเฟสของ SP MOTOR แบบ PM ทั้งชนิดที่เป็น COMMON ภายนอกและชนิดที่เป็น COMMON ภายใน ใช้หลักการเดียวกัน

2.3 การประมวลผลภาพเพื่อทำการตรวจสอบการเคลื่อนที่ของวัตถุ มีดังนี้

2.3.1 ใช้วิธีเก็บพื้นหลังไว้ก่อน คือเมื่อมีวัตถุเคลื่อนที่เข้ามาอยู่ในมุมมองก็ใช้ พื้นหลังที่เก็บไว้ก่อนมาเปรียบเทียบกับเพื่อหาตำแหน่ง ของวัตถุที่เคลื่อนที่

2.3.2 ใช้ Block-Matching Algorithm สำหรับการประมาณการเคลื่อนที่ของวัตถุ จะเห็นดังรูปที่ 2.14 หลักการคือจะหาส่วนที่เข้ากันได้ (Match) ของมาโครบล็อก(Macroblock) ในเฟรมปัจจุบันกับ เฟรมอ้างอิง ซึ่งอาจจะเป็นเฟรมก่อนหน้า โดยจะเข้าไปค้นหาในพื้นที่ ที่เรียกว่า พื้นที่ค้นหา (Search Area) หรือวินโดว์ค้นหา (Search Window) เมื่อเจอส่วนที่เข้ากันได้แล้ว ระยะและทิศทางของตำแหน่งส่วนที่เจอกับตำแหน่งของ มาโครบล็อก คือ เวกเตอร์ของการเคลื่อนที่



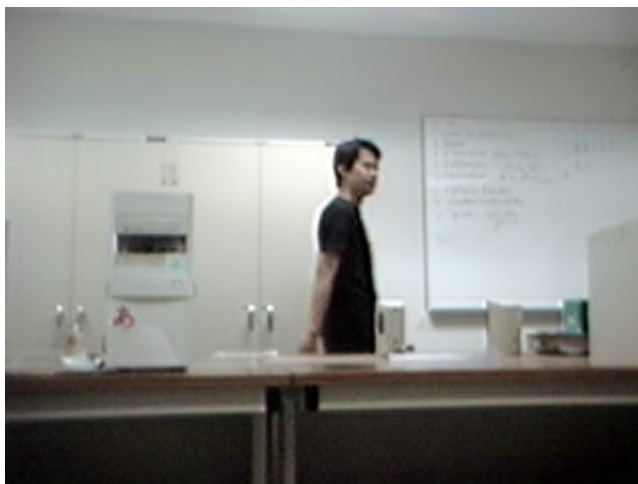
รูปที่ 2.14 แสดงการใช้มาโครบล็อกจากเฟรมปัจจุบันเข้าไปค้นหาส่วนที่เข้ากันได้ภายในวินโดว์ค้นหา ภายในเฟรมอ้างอิง จนเจอส่วนที่ เข้ากันได้แล้วก็จะได้เวกเตอร์ของการเคลื่อนที่

2.3.3 วิธีที่ประยุกต์ขึ้นมา ซึ่งเป็นวิธีที่ใช้ในการตรวจสอบการเคลื่อนที่ของวัตถุในระบบนี้

วิธีนี้อาศัยข้อมูลภาพการเคลื่อนที่ของวัตถุ 3 เฟรมด้วยกัน จะให้เป็น ได้แก่ เฟรมที่ F_t , F_{t-1} และ F_{t-2} โดยที่ F_t เป็นเฟรมปัจจุบัน, F_{t-1} เป็นเฟรมก่อนหน้า F_t และ F_{t-2} เป็นเฟรมก่อนหน้าเฟรม F_{t-1} ดังตัวอย่างเช่นรูปที่ 2.15, 2.16 และ 2.17 สำหรับ F_{t-2} , F_{t-1} และ F_t ตามลำดับ

ขั้นตอนการตรวจสอบการเคลื่อนที่ของวัตถุ ประกอบด้วย

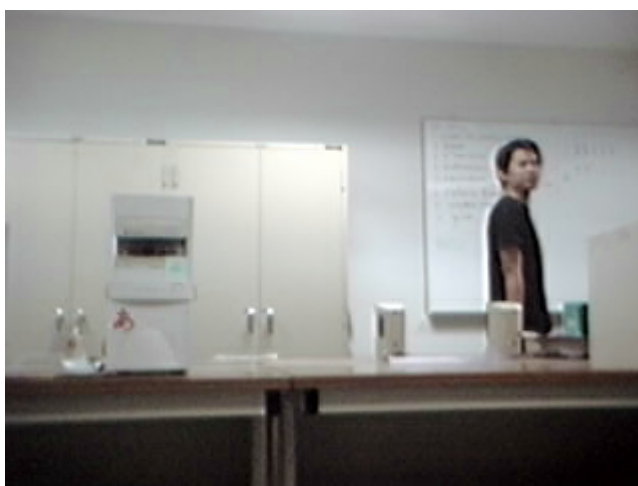
1. สร้างสี่เหลี่ยมกรอบบริเวณที่ไม่เหมือนกันของ F_{t-2} กับ F_{t-1} ดังรูปที่ 2.18
2. สร้างสี่เหลี่ยมกรอบบริเวณที่ไม่เหมือนกันของ F_{t-1} กับ F_t ดังรูปที่ 2.19
3. นำสี่เหลี่ยมที่ได้ทั้งสองมาตรวจสอบหาบริเวณที่ทับกัน สี่เหลี่ยมที่ได้จากส่วนที่ทับกันนี้คือสี่เหลี่ยมที่กรอบวัตถุที่เคลื่อนที่อยู่นั่นเอง ดังรูปที่ 2.20
4. นำสี่เหลี่ยมที่ได้จากข้อ 3 มาวางในมุมมองกันข้ามใน สี่เหลี่ยมที่ได้จากข้อ 2 ก็จะได้ตำแหน่งและขนาดของวัตถุโดยประมาณในเฟรมปัจจุบัน(F_t) ดังรูปที่ 2.21



รูปที่ 2.15 แสดงเฟรม Ft-2



รูปที่ 2.16 แสดงเฟรม Ft-1



รูปที่ 2.17 แสดงเฟรม Ft



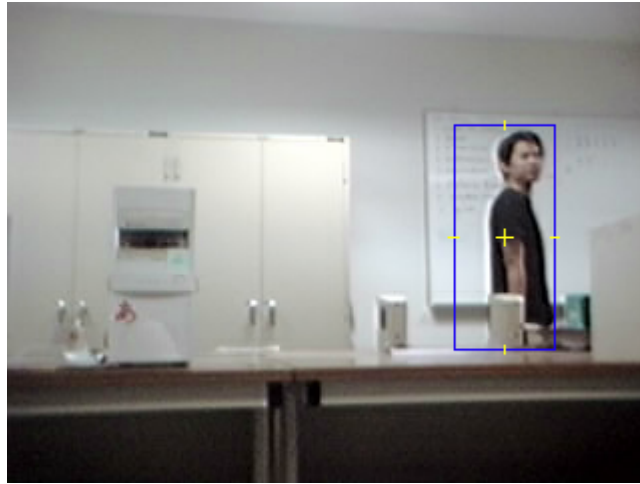
รูปที่ 2.18 แสดงสี่เหลี่ยมกรอบบริเวณที่ไม่เหมือนกันของเฟรม Ft-2 กับ Ft-1



รูปที่ 2.19 แสดงสี่เหลี่ยมกรอบบริเวณที่ไม่เหมือนกันของเฟรม Ft-1 กับ Ft

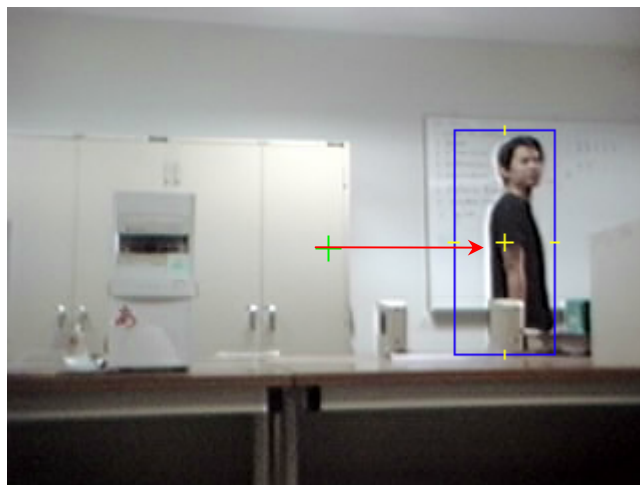


รูปที่ 2.20 แสดงสี่เหลี่ยมกรอบส่วนที่ทับกัน ดังเส้นสีน้ำเงิน



รูปที่ 2.21 แสดงการนำสี่เหลี่ยมที่ได้จากรูปที่ 2.20 มาวางในเฟรม Ft
ซึ่งก็จะได้ขนาดและตำแหน่งของวัตถุที่เคลื่อนที่อยู่

เมื่อได้ตำแหน่งและขนาดของวัตถุในเฟรมปัจจุบันแล้วก็สามารถคำนวณหาจุดศูนย์กลางของวัตถุนั้นได้ ซึ่งเมื่อนำมาเทียบกับจุดศูนย์กลางของภาพแล้ว ก็จะได้เวกเตอร์ที่เป็นข้อมูลเพื่อที่ใช้ในการบังคับให้อุปกรณ์เคลื่อนที่ตามวัตถุ เพื่อให้วัตถุนั้นอยู่ในมุมมองของกล้องตลอดเวลา ดังรูปที่ 2.22



รูปที่ 2.22 แสดงเวกเตอร์ที่ใช้เป็นข้อมูลในการบังคับอุปกรณ์ให้ทำการหมุนเคลื่อนที่ตามวัตถุ